Copyright 2024–2026 Strawberry Creek Monitoring Group contributors.
Licensed under the Apache License, Version 2.0.
https://www.apache.org/licenses/LICENSE-2.0

The Mathematics and Design of DuskCrayfish

Technical Report

Daniel Boupha Christensen **Anisa Khair** **Ian Liu**
Environmental, Health, and Safety, University of California, Berkeley
The Green Initiative Fund

August 14, 2026

Abstract

DuskCrayfish learns what a small urban creek normally looks like and then flags moments it cannot predict well. It takes a window of recent sensor history across all the connected monitoring sites and predicts what the very next reading should be at every site; when the real reading and the prediction diverge sharply on conductivity, that gap is the anomaly signal. We never tell the model what a discharge looks like. It only ever learns normal, and an anomaly is whatever it fails to reproduce. This document sets out the mathematics of the deployed model, that is, a graph convolution over the creek's flow topology followed by a per-node recurrent forecaster, together with the detection rules we built on top of it and the reasoning behind each choice. We kept the architecture small, and we give our reasons for this alongside the mathematics.

1 Introduction

DuskCrayfish learns what the creek normally looks like and then flags moments it cannot predict well. Everything below follows from one idea: we take a window of recent sensor history across all the connected sites, and we predict what the very next reading should be at every site. When the real reading and the prediction diverge sharply on conductivity, that gap is the anomaly signal. We never tell the model what a discharge looks like. It only ever learns normal, and an anomaly is whatever it fails to reproduce.

This document explains the math and, alongside each piece, why we built the model the way we did. The architecture is deliberately small, and we explain what led us to that at each point where the choice arises.

1.1 The deployed configuration

The values in Table 1 are the actual settings the model runs with, drawn from the project's configuration. We refer back to them throughout the mathematics that follows so the general equations stay tied to this specific model.

One graph layer, one LSTM layer, and a hidden dimension of only 16 is a deliberately lean model. The training data is a rolling window of recent creek behavior, not a vast historical archive, and a larger model on that much data would overfit, memorizing the recent past rather than learning the creek's general rhythm. A small model also trains in seconds on a CPU, which matters because the data is a moving window and we retrain the model regularly to keep it current rather than training it once and freezing it. We sized the model to the data that actually exists, and not to what a larger model would suggest on paper.

Table 1: The deployed configuration. These are the settings the current checkpoint runs with, not defaults or recommendations.

Setting	Value
Sequence length T	24 timesteps (6 hours at 15 min sampling)
Features per node F	10 (3 sensor, 3 weather, 4 cyclical time)
Hidden dimension d	16
Graph convolution layers	1
LSTM layers	1
Graph convolution type	GCN
Dropout	0.1
Training epochs	10 (early stopping, patience 5)
Batch size	128
Learning rate	0.001
Train / validation split	0.8 / 0.2
Threshold percentile	99
Rain lookback window	12 hours
Rain threshold multiplier m	2.0
Rain amount that counts as wet θ	0.1
Post rain decay window	36 hours
Imputation gap limit	24 hours

2 Data and Notation

We represent the creek as a graph with N sensor nodes whose connections follow the direction water flows. At each timestep every node carries a vector of features: conductivity, depth, and temperature from the sensors, air temperature, rainfall, and solar radiation merged in from weather data when it is available, and four cyclical time encodings, i.e., the sine and cosine of hour of day and of day of year. We did not hardcode the number of features. The model reads the feature count from the data it is given at runtime and sizes its input and output layers to match, which is why it adapts cleanly when a weather channel is missing. We write F for that count and treat $F = 10$ as the deployed case throughout. Table 2 collects the symbols used in the rest of this document.

A single training example is a window of $T = 24$ consecutive timesteps,

$$\mathbf{X} \in \mathbb{R}^{T \times N \times F}, \tag{1}$$

where $\mathbf{X}_t \in \mathbb{R}^{N \times F}$ is the snapshot of every node's features at timestep t. The target is the snapshot immediately after the window:

$$\mathbf{Y} = \mathbf{X}_{T+1} \in \mathbb{R}^{N \times F}. \tag{2}$$

We chose $T = 24$ because it is six hours of history at the creek's fifteen minute cadence. That is long enough to capture the daily rhythms the creek actually has (e.g., the warming through the afternoon, the overnight runoff from sprinklers) without reaching so far back that old conditions drown out the recent trend the model is trying to extend.

Before anything else, we normalize every feature to a z-score using statistics from complete data only:

$$z = \frac{x - \mu}{\sigma}. \tag{3}$$

This choice has a consequence that the model relies on. After normalization a value of zero is the average, that is, the neutral middle of the distribution. That is exactly the placeholder we use for missing sensor readings, so a gap contributes nothing rather than dragging the model up or down. This is what lets the model run on a network where some sites are offline: a sensor that is down simply reads as the neutral zero everywhere it is missing.

Table 2: Notation.

Symbol	Meaning	Dimension
N	number of sensor nodes	scalar, $N = 4$
F	features per node	scalar, $F = 10$
T	timesteps in one input window	scalar, $T = 24$
d	hidden dimension	scalar, $d = 16$
d_e	node embedding dimension	scalar, $d_e = 8$
$\mathbf{X}$	one input window	$\mathbb{R}^{T\times N\times F}$
$\mathbf{X}_t$	feature snapshot at timestep t	$\mathbb{R}^{N\times F}$
$\mathbf{Y}$	target snapshot, the step after the window	$\mathbb{R}^{N\times F}$
$\hat{\mathbf{Y}}$	predicted snapshot	$\mathbb{R}^{N\times F}$
$\mathbf{A}$	directed flow adjacency	$\{0,1\}^{N\times N}$
$\hat{\mathbf{A}}$	adjacency with self loops, $\mathbf{A}+\mathbf{I}$	$\{0,1\}^{N\times N}$
$\hat{\mathbf{D}}$	diagonal degree matrix of $\hat{\mathbf{A}}$	$\mathbb{R}^{N\times N}$
$\mathbf{H}$	node features at one timestep	$\mathbb{R}^{N\times F}$
$\mathbf{H}'$	node features after graph convolution	$\mathbb{R}^{N\times d}$
$\mathbf{W}$	graph convolution weights	$\mathbb{R}^{F\times d}$
$\mathbf{h}_t$	pooled creek summary at timestep t	$\mathbb{R}^{d}$
$\mathbf{e}_i$	learned embedding of node i	$\mathbb{R}^{d_e}$
$\mathbf{z}_{i,t}$	temporal model input for node i	$\mathbb{R}^{2d+d_e}$
$\mathbf{h}_{i,T}^{\mathrm{lstm}}$	final recurrent state for node i	$\mathbb{R}^{d}$
$\mathbf{W}_{\mathrm{out}}, \mathbf{b}_{\mathrm{out}}$	output layer	$\mathbb{R}^{F\times d}$, $\mathbb{R}^{F}$
e_{ij}	prediction error, node i feature j	scalar
s_i	normalized anomaly score at node i	scalar
τ_i	alerting threshold at node i	scalar
m, θ, D	rain multiplier, wet threshold, decay window	scalars
k	level shift threshold in robust deviations	scalar, $k = 4.0$

2.1 The deployed four-node core

It is worth making the graph concrete for the network the model actually runs on. The four active nodes, in the order the code indexes them, are north fork 0, south fork 2, south fork 1, and oxford. We connect them with directed edges following the physical direction of water flow downhill. There are three of them:

$$\text{north fork } 0 \to \text{oxford}, \quad \text{south fork } 1 \to \text{south fork } 2, \quad \text{south fork } 2 \to \text{oxford}. \tag{4}$$

Writing these as a directed adjacency $\mathbf{A}$, where entry (i, j) is one when there is an edge from node i to node j, in the code's index order (north fork 0, south fork 2, south fork 1, oxford) gives

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}. \tag{5}$$

Reading the rows of Equation 5: north fork 0 flows to oxford, south fork 2 flows to oxford, south fork 1 flows to south fork 2, and oxford flows nowhere, since it is the most downstream point. That last empty row is the signature of a sink, i.e., the confluence where the network ends. The graph convolution then adds a self loop to every node and applies the degree normalization described in Section 3.1, so each node's update blends its own value with the values flowing into it along these edges. The structure is sparse and entirely determined by the creek's physical layout, which means the model does not have to "learn" who connects to whom. We supply the connectivity directly, and it is correct by construction.

2.2 Why footbridge is not a node

An earlier version of this network had five nodes, with footbridge sitting between north fork 0 and oxford on the north branch. We removed footbridge and contracted its edge, so north fork 0 now flows directly to oxford. The water still runs that way; there is simply no sensor in between.

The reason is a mismatch between training and deployment. Inside the corpus window footbridge has 1,191 real readings against 29,123 timesteps, i.e., 4.1 percent real coverage, and it does not appear in the live data cache at all. Everything else was imputation. A node that is almost entirely fabricated in training and entirely absent in production teaches the model to lean on a signal that will not be there when it runs, and it fed that fabricated signal to its graph neighbours as though it were real. Contracting the edge keeps the flow topology honest and trains the model on the same set of sites it serves.

3 Model Architecture

3.1 The spatial step, a graph convolution

At each timestep we let each node's features be informed by the nodes connected to it, because a disturbance upstream shows up downstream as the water carries it. With $\mathbf{H} \in \mathbb{R}^{N \times F}$ the node features at one timestep, the graph convolution [Kipf and Welling, 2017] computes:

$$\mathbf{H}' = \mathrm{ReLU}\Big(\hat{\mathbf{D}}^{-\frac{1}{2}}\,\hat{\mathbf{A}}\,\hat{\mathbf{D}}^{-\frac{1}{2}}\,\mathbf{H}\,\mathbf{W}\Big), \tag{6}$$

where $\mathbf{W} \in \mathbb{R}^{F \times d}$ is a learned weight matrix projecting the F features into the hidden size $d = 16$, and the nonlinearity is

$$\mathrm{ReLU}(x) = \max(0, x). \tag{7}$$

This is the standard graph convolutional operator. The adjacency with inserted self loops is $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, and $\hat{\mathbf{D}}$ is its diagonal degree matrix with $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$. The self loops let each node keep its own features rather than only seeing neighbors, and the degree scaling $\hat{\mathbf{D}}^{-1/2}$ on both sides keeps a heavily connected site from swamping a lightly connected one. The graph library [Fey and Lenssen, 2019] adds the self loops and applies this normalization automatically, so the edge list we give the model does not need to include them.

The matrix form in Equation 6 is compact, but it hides what happens to one site. Written per node, the new representation of node i is a normalized sum over itself and the nodes connected to it:

$$\mathbf{h}'_i = \mathrm{ReLU}\left(\sum_{j \,\in\, \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i\,\hat{d}_j}}\,\mathbf{h}_j\,\mathbf{W}\right), \tag{8}$$

where $\mathcal{N}(i)$ is the set of nodes connected to i through the graph and $\hat{d}_i$ is node i's degree including its self loop. The intuition is as follows: each node's updated features are a degree-weighted blend of its own and its neighbors', passed through the learned projection $\mathbf{W}$ and the nonlinearity. This is what it means to borrow from your neighbors. A spike in conductivity at a site therefore enters the connected site's representation on the very next layer, which is exactly the propagation we need to recognize a real event as it moves through the creek.

Two design choices live here. The first is the type of convolution: we set the GCN type explicitly in the configuration, and a GCN weights neighbors by the fixed graph structure, which is the right prior when the connections are real water flow rather than relationships the model should infer. One way to do this would be to let the model learn its own neighbor weightings instead. However, the flow topology is known physics, so there is little reason to make the model learn weightings it would only overfit on limited data. The second choice is depth: we use a single graph layer, which means each node sees its immediate neighbors. On a network this small, with short flow paths, one hop already reaches the sites that matter, and stacking more layers would blur every node toward a single averaged value, a known failure of deep graph networks called oversmoothing [Li et al., 2018].

3.2 Pooling, collapsing the network into one creek state

The temporal model wants a single vector summarising the whole creek at each timestep alongside the per-node detail, so we average across the nodes:

$$\mathbf{h}_t = \frac{1}{N}\sum_{i=1}^{N} \mathbf{H}'_i \ \in \ \mathbb{R}^d. \tag{9}$$

Doing this at every timestep produces a sequence $\mathbf{h}_1, \ldots, \mathbf{h}_T$, each entry a summary of the whole connected network at one moment. We use mean pooling rather than a more elaborate combination because it is the simple, robust choice, and it interacts well with the missing-data handling: when a node mask is supplied we take the average over the reporting sites only, so the creek summary reflects real data rather than placeholder zeros.

The pooled summary does not replace the per-node representations, it accompanies them. An earlier version pooled and discarded $\mathbf{H}'$, which left the temporal model with no way to tell one site from another and made every node receive the same prediction. We carry both forward to the next step.

3.3 The temporal step, an LSTM

The sequence goes into a single layer Long Short Term Memory network [Hochreiter and Schmidhuber, 1997], which tracks how the signal evolves and carries relevant information forward across steps. At each timestep it maintains a hidden state $\mathbf{h}_t^{\mathrm{lstm}}$ and a cell state $\mathbf{c}_t$, updated through gates:

$$\begin{aligned}
\mathbf{f}_t &= \sigma\big(\mathbf{W}_f \mathbf{h}_t + \mathbf{U}_f \mathbf{h}_{t-1}^{\mathrm{lstm}} + \mathbf{b}_f\big)\,, \\
\mathbf{i}_t &= \sigma\big(\mathbf{W}_i \mathbf{h}_t + \mathbf{U}_i \mathbf{h}_{t-1}^{\mathrm{lstm}} + \mathbf{b}_i\big)\,, \\
\mathbf{o}_t &= \sigma\big(\mathbf{W}_o \mathbf{h}_t + \mathbf{U}_o \mathbf{h}_{t-1}^{\mathrm{lstm}} + \mathbf{b}_o\big)\,, \\
\tilde{\mathbf{c}}_t &= \tanh\big(\mathbf{W}_c \mathbf{h}_t + \mathbf{U}_c \mathbf{h}_{t-1}^{\mathrm{lstm}} + \mathbf{b}_c\big)\,, \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\
\mathbf{h}_t^{\mathrm{lstm}} &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned} \tag{10}$$

The three gates are each squashed by the sigmoid σ into values between zero and one, acting as soft switches. The symbol $\odot$ is elementwise multiplication, and the $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{b}$ are learned. The intuition behind this network is that the forget gate $\mathbf{f}_t$ decides how much of the previous memory to keep, the input gate $\mathbf{i}_t$ how much of the new candidate $\tilde{\mathbf{c}}_t$ to store, and the output gate $\mathbf{o}_t$ how much of the resulting memory to emit. The cell update line is where the network actually remembers or forgets.

We chose the LSTM over a plain recurrent network because its gates let it hold a trend from hours earlier when relevant and discard it when not, which matters for a six hour window. We use a single LSTM layer for the same reason the rest of the model is small: the sequences are short and the data is limited, so one layer captures the temporal pattern without overfitting. We keep the final hidden state,

$$\mathbf{h}_T^{\mathrm{lstm}} \in \mathbb{R}^d, \tag{11}$$

which is the model's compressed "understanding" of how the creek has been behaving over the whole window.

3.4 The output, predicting the next reading

The temporal model runs per node rather than once for the whole creek. For node i at timestep t the input is that node's own graph convolution output, the pooled creek summary, and a learned embedding of the node's identity, concatenated:

$$\mathbf{z}_{i,t} = \big[\, \mathbf{H}'_{i,t} \parallel \mathbf{h}_t \parallel \mathbf{e}_i \,\big] \in \mathbb{R}^{2d+d_e}, \tag{12}$$

with $\mathbf{e}_i \in \mathbb{R}^{d_e}$, $d_e = 8$, learned one vector per site. The node embedding is what lets the model "know" that south fork 2 normally runs hotter than north fork 0 rather than treating every site as interchangeable. The

sequence $\mathbf{z}_{i,1}, \ldots, \mathbf{z}_{i,T}$ goes through the LSTM of Equation 10, and the final hidden state $\mathbf{h}_{i,T}^{\text{lstm}} \in \mathbb{R}^d$ is that node's own compressed history.

The output layer predicts a change from the last observed reading rather than an absolute value:

$$\hat{\mathbf{Y}}_i = \mathbf{X}_{T,i} + \left(\mathbf{W}_{\text{out}}\, \mathbf{h}_{i,T}^{\text{lstm}} + \mathbf{b}_{\text{out}}\right), \tag{13}$$

with $\mathbf{W}_{\text{out}} \in \mathbb{R}^{F \times d}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^F$ learned, and $\mathbf{X}_{T,i}$ the node's features at the last timestep of the window, feature-propagated if a mask filled it in. Everything is in normalized units, so no denormalization happens here.

The residual form in Equation 13 is the reason the model has any notion of where things currently are. One way to build the output would be to predict from the hidden state alone. However, the model then falls back on whatever average shape minimizes error across the training set, which is climatology rather than a forecast.

Both choices show up in the predictions. On the deployed checkpoint, the spread across nodes, that is, the largest node prediction minus the smallest, reaches 4.592 in normalized units. Decomposing it, the learned head contributes 0.188 and the residual anchor contributes 4.591. Concretely, almost all of the per-node difference comes from each node starting at its own last reading, and the head supplies a comparatively small correction on top. A version without node identity would give every node the same number, i.e., a spread of exactly zero.

4 Training

We train by minimizing the mean squared error between prediction and truth,

$$\mathcal{L} = \frac{1}{|\mathcal{M}|\,|\mathcal{S}|} \sum_{(i)\in\mathcal{M}} \sum_{j\in\mathcal{S}} \left(\hat{\mathbf{Y}}_{ij} - \mathbf{Y}_{ij}\right)^2, \tag{14}$$

over training data that contains only ordinary creek behavior. The model learns the absence of discharges, not discharges themselves.

Two restrictions narrow that sum. $\mathcal{S}$ is the scored channel set, that is, conductivity, depth and temperature, not all F channels. The model still predicts every channel it was given. However, rainfall 15 minutes ahead is not learnable from creek sensors, and scoring it adds irreducible error that is worst during storms, which is exactly when conductivity matters most. $\mathcal{M}$ is the set of node targets that were actually observed and passed quality control. A target that was never read is filled with the neutral zero, and scoring the model against that fabricated zero teaches it to reproduce an absence.

Two further training choices are worth naming. We apply dropout of 0.1 during training, randomly zeroing a small fraction of hidden units each step, which prevents the small model from leaning too hard on any single pattern. Training runs for up to 10 epochs with early stopping at patience 5. That is, if the validation loss stops improving for five epochs the run halts and the best weights are restored, so the model does not train past the point where it is still learning something general. We train in plain 32 bit precision on CPU, because the mixed precision that speeds up large models on a graphics card actually breaks the LSTM computation on a CPU and offers no benefit at this size.

5 Detection

5.1 Model all, alert one

Once trained, the model runs on new data and we examine where it missed. The prediction error at node i on feature j is the absolute deviation:

$$e_{ij} = \left|\hat{\mathbf{Y}}_{ij} - \mathbf{Y}_{ij}\right|. \tag{15}$$

The defining choice is which of those errors we act on. The model predicts all features, but we use only conductivity error to decide whether something is anomalous. This is the model all, alert one strategy.

Conductivity is the channel that actually responds to discharges, while the others, rainfall especially, produce large prediction errors that have nothing to do with water quality. An earlier version that scored on the average error across all features was effectively flagging rainstorms, because the rain prediction error swamped everything. Isolating conductivity removed those false alarms.

We keep the score per node rather than averaging it across the network. Writing c for the conductivity index, node i's score at a timestep is e_{ic}, and we judge it against that node's own history rather than a shared bar:

$$s_i = \frac{e_{ic} - \text{median}_i}{\text{IQR}_i}, \tag{16}$$

where median_i and IQR_i are the median and interquartile range of node i's conductivity error over held out validation data. Averaging across nodes was the earlier design, on the reasoning that an event propagating through the network should score higher than a single site blip. We replaced it because sites differ: one that simply runs noisier than the rest sits permanently above a shared threshold, and averaging lets a large deviation at one node be diluted by three quiet ones. Robust normalization per node fixes both, and keeping the node axis is what lets an alert say which site is affected.

The threshold is also per node, fixed once during training as the 99th percentile of that node's normalized validation errors:

$$\tau_i = \text{percentile}_{99}\left(\{s_i^{\text{val}}\}\right). \tag{17}$$

We fix it at training time rather than recomputing it on whatever data is in front of the model, and that is deliberate: recomputing would be circular and would let the threshold drift with each batch. The 99th percentile means that under normal conditions only about one percent of timesteps at a node would cross it, which sets the false alarm rate before deployment rather than discovering it after. A single timestep over the bar is not an alert. A node is flagged only when at least three scored timesteps in the window cross it, so one noisy reading cannot raise an alarm.

We compute median, interquartile range and threshold on observed targets only. A node target that was never read is filled with the neutral zero, and letting those into the calibration sets the bar against data that does not exist. This matters in practice: on the current corpus one site had 27.7 percent of its target cells fabricated, and its threshold was calibrated almost entirely against the error of predicting a zero.

5.2 The second rule, sustained level shift

The error based rule above is structurally blind to one shape. A clean step in conductivity "surprises" the model once and is then absorbed into its own forecast, so the residual goes quiet while the water stays abnormal. A second rule watches the level itself rather than the error:

$$u_i = \frac{\mathbf{Y}_{ic} - \text{median}_i^{\text{lvl}}}{\text{IQR}_i^{\text{lvl}}}, \qquad |u_i| > k, \tag{18}$$

with $k = 4.0$ and the median and interquartile range taken over the node's normalized conductivity level on validation data. A node is anomalous if either rule fires, and the result records which one did, since an onset and a sustained abnormal level mean different things operationally.

5.3 The rain adjustment

Rain genuinely raises conductivity without being a discharge, as runoff carries road salt and grime into the creek. To avoid flagging ordinary rain, we raise the threshold during wet periods. Summing rainfall over the preceding 12 hour window, the per timestep threshold is

$$\tau_t = \begin{cases} m \cdot \tau & \text{if } \sum_{k \in \text{window}} r_k > \theta, \\ \tau\left(1 + (m-1)\left(1 - \text{clip}\left(\frac{\Delta_t}{D}, 0, 1\right)\right)\right) & \text{otherwise, if it rained within } D \text{ hours}, \\ \tau & \text{otherwise}, \end{cases} \tag{19}$$

where r_k is the rainfall at timestep k, $\theta = 0.1$ the small amount that counts as wet, $m = 2.0$ the multiplier, $D = 36$ hours the post rain decay window, and Δ_t the hours elapsed since the lookback window last closed on wet weather. That is, we double the bar while rain is recent, and then ease it back to normal over a day and a half rather than dropping it in one step. The 12 hour window and the doubling are the tuned values. They suppress the routine first flush of a storm while still leaving room to catch a deviation large enough to clear twice the normal bar.

The taper exists because the step alone had a gap. With a hard edge the threshold returns to normal the moment the lookback window clears, so a delayed first flush pulse arriving a few hours later meets the ordinary bar and is flagged as a discharge. The decay covers that window and returns smoothly instead.

There is a limit in the other direction. A genuine discharge that happens to occur during or shortly after rain has to clear a bar that is up to twice as high, so real events in wet weather are harder to detect, and one confirmed event in the catalog is missed for exactly this reason. Where to set m and D is a trade between missing discharges in the wet and flagging ordinary runoff, and it is a judgment about what the system should alarm on rather than a flaw in the math.

6 Limitations and Future Work

Nearly every choice in the design points the same direction: keep the model small and matched to limited, rolling data, encode the known physics of water flow directly into the graph rather than making the model learn it, and decide anomalies on the one channel that actually carries the signal.

Three of the limitations named in an earlier version of this document have since been addressed, and it is worth saying which. We added cyclical time features, so the model now has hour of day and day of year context and no longer reads an out of season window as unusual everywhere. The loss scores conductivity, depth and temperature rather than conductivity alone. And we widened the post rain window into the decay described in Section 5.3. Those were listed as natural next steps and we took them.

What remains is a shorter list. The first two are the most important.

Scoring on conductivity alone means an anomaly that shows up only in another channel (e.g., a dissolved oxygen crash from an oil sheen) would not be caught by the current alarm. The loss now watches three channels, but detection still watches one.

The rain adjustment gives up some detection in order to suppress false alarms. A discharge during or shortly after rain has to clear up to twice the ordinary bar, and at least one confirmed event in the catalog is not detected under the deployed multiplier. Lowering it would recover that event at the cost of flagging ordinary runoff. We chose these values by judgment rather than deriving them, and they belong to whoever owns the alerting budget.

The network is four nodes, not five. We dropped footbridge for the coverage reason given in Section 2.2, which keeps training and deployment consistent but leaves the north branch with no sensor between north fork 0 and oxford. An event entering there is seen only once it reaches oxford.

The threshold rests on a single chronological split of a corpus that begins in June 2025. The held out portion covers a season the training portion does not contain, so distribution shift and overfitting cannot be fully separated with the data as it stands. We leave a same season holdout, which would separate them, as an open question for future work.

Finally, we retrain the model on a rolling window rather than training it once and freezing it, so every number in this document describes the current checkpoint rather than the architecture in perpetuity. The thresholds in particular are fitted quantities and will move when the model is refitted.

References

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. URL `https://arxiv.org/abs/1609.02907`.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. URL `https://arxiv.org/abs/1903.02428`.

Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, 2018. URL `https://arxiv.org/abs/1801.07606`.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.